Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

Agentic AI Applications with Claude Code

TGS Ref No: TGS-2025052468

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.7

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	27 Jun 2026	First version — Topics 1 –3 (Claude Code fundamentals, tools & commands, skills/agents/hooks), step-by-step activities and command reference	Dr. Alfred Ang
1.1	27 Jun 2026	Added Claude product family, AI-engineering evolution timeline, harness engineering, pricing, the .claude directory, hooks automation, sub-agents rationale, and the assessment flow & LMS details	Tertiary Infotech Academy Pte Ltd
1.2	27 Jun 2026	Added the 7-step build workflow (Activity 1a/1b), use cases & agentic-loop example, CLI-install-via-VS-Code tip, Hooks activity (real CC hooks + form pre/post), and the capstone build-prompt template	Tertiary Infotech Academy Pte Ltd
1.3	2 Sep 2026	Added Claude for Chrome and Claude for Science to the Claude product family; added the Plan step to both agentic-loop lists; added the SKILL.md vs CLAUDE.md distinction to Topic 3.1.	Tertiary Infotech Academy Pte Ltd
1.4	2 Sep 2026	Reworked Activity 6 from form pre/post hooks to a floating WhatsApp widget with a 10-second auto-open hook, suggested queries and voice activation.	Tertiary Infotech Academy Pte Ltd
1.5	4 Sep 2026	Added a user-level vs project-level comparison (commands, MCP tools, skills,	Tertiary Infotech Academy Pte Ltd

		CLAUDE.md, hooks) to section 2.0 The .claude Directory, aligned to the new slide 42 of the 68-slide deck (v14.0)	
1.6	9 Sep 2026	Added a “Key Concepts Summary — What Triggers What” section covering CLAUDE.md, skills, tools, slash commands, hooks and sub agents and their triggers; expanded 3.1 with a SKILL.md vs CLAUDE.md comparison table.	Tertiary Infotech Academy Pte Ltd
1.7	9 Sep 2026	Added the Lab Materials section describing the four restructured lab folders. Fixed table column widths so all comparison tables render both columns.	Tertiary Infotech Academy Pte Ltd

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

How to Use This Guide

This learner guide follows the three topics of the course and contains **step-by-step instructions for every hands-on activity**. Keep it open beside Claude Code as you work. During the open-book assessment you may refer to this guide, the slides and any approved materials.

By the end of the course you will have built, deployed, tested, secured and optimised a real software-development company website — entirely with Claude Code.

What you need before you start:

- A computer with internet access (macOS, Linux or Windows)
- Claude Code installed and signed in
- A GitHub account
- Node.js installed (for `npx` commands)

Topic 1 — Claude Code Fundamentals

1.1 Overview of Claude Code

Claude Code is an **agentic coding tool** that runs in your terminal. Instead of autocompleting code line by line, you give it a goal in plain English and it plans the work, edits files, runs commands, uses tools, and verifies the result.

Key ideas:

- **Terminal-native** — runs where you already work.
- **Agentic** — plans and executes multi-step tasks.
- **Tool-using** — reads/writes files, runs bash, calls MCP servers.
- **Extensible** — custom commands, skills, sub agents and hooks.

The Claude product family

Anthropic offers several Claude products — all powered by the same Claude models, differing in where they run and who they're for:

- **Claude.ai** — the chat assistant on web, desktop and mobile for everyday tasks.
- **Claude Code** — agentic coding tool in your terminal, IDE and desktop.
- **Claude Cowork** — desktop agent for non-developers to automate files and tasks.
- **Claude Design** — generates polished UI and frontend designs.
- **Claude for Microsoft** — Claude inside Microsoft 365 and Outlook.
- **Claude for Chrome** — browser agent that navigates and acts on the web for you.
- **Claude for Science** — speeds up research workflows for scientists and labs.

The evolution of AI engineering (2023 → 2026)

As models grow more capable, the work shifts from wording a single prompt to engineering the whole system around the model:

- **2023 — Prompt Engineering:** craft the wording of a single prompt to get good output.
- **2024 — Context Engineering:** curate what the model sees — files, memory and tools.

- **2025 — Harness Engineering:** build the system around the model — loop, tools, verification.
- **2026 — Harness Systems:** agents run long tasks autonomously (Claude Code, Cowork, Codex).

What Claude Code can do

Claude Code handles real engineering tasks end to end — you describe the outcome and it does the work: **build** features and whole apps, **debug & fix** errors, **refactor & improve** existing code, and **automate** git, tests, deployment and CI/CD. Common use cases: marketing sites and landing pages, internal web tools, full-stack apps, mobile apps, scripts and automations, and data tasks.

The agentic loop in action

Example — you ask: *"Build a responsive software-dev website with an enquiry form, then test it."*

1. **Gather context** — reads the folder and CLAUDE.md to understand the existing site and conventions.
2. **Plan** — drafts the approach (hero, testimonials and enquiry form sections) for you to approve.
3. **Take action** — writes `index.html`, `styles.css` and `script.js` with form validation.
4. **Verify work** — opens the page via the Playwright MCP and submits the form to test it.
5. **Repeat** — spots a validation bug, fixes `script.js`, re-tests, then reports done.

What is harness engineering?

A **harness** is the system built around the model — the agentic loop, tool access, context and memory management, and verification and guardrails — that turns a capable model into an agent which can complete long-running tasks. **Harness engineering** is designing that scaffolding (and stripping it back as models improve). Examples of harness systems include **Claude Code, Claude Cowork, OpenAI Codex, Gemini CLI, OpenClaw and Hermes Agent**. The guiding principle: build the simplest harness that works, and re-examine it with every new, more capable model.

1.2 The Agentic Loop

Every task Claude Code performs runs the same repeating cycle:

1. **Gather context** — read files, run searches, use tools to understand the task.
2. **Plan** — decide the approach and, for bigger tasks, confirm the plan before making changes.
3. **Take action** — edit code, run commands, call tools to make progress.
4. **Verify work** — run tests, lint, view output to check the result is correct.
5. **Repeat** — feed results back as new context and loop until the goal is met.

Why it matters:

- Give *direction*, not micro-steps — state the goal and constraints clearly.
- *Verification is everything* — give Claude a way to check its own work (tests, linters, screenshots).
- *Steer mid-loop* — review the plan, interrupt and course-correct.
- *Context is the fuel* — what Claude can see determines what it can do.

1.3 Context Engineering

Context engineering is curating exactly what Claude sees so it has the right information and nothing that wastes its limited context window.

Tool	Purpose
`CLAUDE.md`	Project memory loaded every session. Generate with `/init`.
@ mentions	Pull specific files or folders into context on demand.
`/compact`	Summarise the conversation to reclaim context space.
`/clear`	Wipe the slate for a fresh, unrelated task.

Best practices: keep `CLAUDE.md` short and high-signal; pull in only what's needed with `@file`; `/compact` at natural breakpoints; `/clear` between unrelated tasks.

1.4 Setting Up Claude Code

Claude Code runs in three interfaces — the same agentic loop everywhere:

- **Terminal (CLI):** install once, then run `claude` in any project folder.
  - macOS/Linux: `curl -fsSL https://claude.ai/install.sh | bash`
  - Windows (PowerShell): `irm https://claude.ai/install.ps1 | iex`
- **IDE (VS Code):** install the Claude Code extension (also works in Cursor and JetBrains). Gives inline diffs, @-mentions with line ranges, and plan review. The extension bundles its own CLI for the chat panel; install the standalone CLI too if you want to run `claude` in the integrated terminal. **Tip:** you don't have to install the CLI by hand — in VS Code just ask Claude Code: *"install claude cli in the terminal for me"*.
- **Desktop app:** a full GUI for parallel sessions, scheduled tasks (Routines), and connectors.

Sign in once with any paid Claude plan (Pro, Max, Team, Enterprise) or a Console account.

Pricing (as of June 2026 — verify at claude.com/pricing): Claude Code is included in subscription plans from **Pro** ($17/mo annual, $20 monthly) upward — **Max** (from $100/mo, 5× or 20× usage), **Team** ($20/seat standard, $100/seat premium), and **Enterprise** ($20/seat + usage at API rates). It's also available **pay-as-you-go via the API**, billed per million tokens (input / output): Opus 4.8 $5 / $25, Sonnet 4.6 $3 / $15, Haiku 4.5 $1 / $5.

1.5 How Claude Code Works — Built-in Tools

Tools are what make Claude Code agentic. Each tool result feeds back into the loop. The built-in tools fall into five categories:

Category	What Claude can do
File operations	Read, edit, create, rename and reorganise files
Search	Find files by pattern, grep content, explore the codebase
Execution	Run shell commands, tests, builds and git
Web	Search the web, fetch docs, look up errors
Code intelligence	See type errors/warnings, jump to definitions (with plugins)

Claude also has tools for spawning subagents and asking you questions. Extend the base set with skills, MCP, hooks and subagents.

1.6 The Context Window

Claude's context window holds the system prompt, `CLAUDE.md`, auto memory, your conversation, file reads, command output, loaded skills, and MCP tool names. As you work, it fills up.

- Run `/context` to see what's using space; `/mcp` for per-server cost.
- Near the limit, Claude **auto-compacts**: it clears old tool outputs first, then summarises the conversation. Detail from early in the chat can be lost — put persistent rules in `CLAUDE.md`.
- MCP tool schemas are deferred (loaded on demand); subagents run in their own context, so they don't bloat yours.

1.7 Memory Files

Each session starts fresh. Two mechanisms carry knowledge across sessions:

- **`CLAUDE.md`** — *you* write it: instructions, conventions, build/test commands. Generate a starter with `/init`. Keep it under ~200 lines. Scopes: project (`./CLAUDE.md` or `./.claude/CLAUDE.md`), user (`~/.claude/CLAUDE.md`), or managed (organisation). Loaded in full every session.
- **Auto memory** — *Claude* writes it: learnings and patterns it discovers. Stored as `MEMORY.md` (plus topic files) per repository under `~/.claude/projects/<project>/memory/`. The first 200 lines / 25KB load each session. Browse and edit with `/memory`.

1.8 Permission Modes

Permission modes control how often Claude pauses to ask. Press **Shift+Tab** to cycle modes (the current mode shows in the status bar).

Mode	Runs without asking
`default`	Reads only — asks before edits and shell commands
`acceptEdits`	Reads, file edits, and common filesystem commands (`mkdir`, `mv`, `cp`…)
`plan`	Reads only — explores and proposes a plan without editing source
`auto`	Everything, with background safety checks (research preview)
`dontAsk`	Only pre-approved tools (for locked-down CI)
`bypassPermissions`	Everything (isolated containers/VMs only)

You can also pre-approve specific commands in `.claude/settings.json`.

1.9 Dangerous Skip Permissions (`bypassPermissions`)

`bypassPermissions` (a.k.a. `--dangerously-skip-permissions`) disables permission prompts and safety checks so tool calls run immediately.

- It offers **no protection** against prompt injection or unintended actions.
- Use it **only** in isolated environments — containers, VMs, dev containers without internet access.
- It refuses to start as root/sudo and cannot be entered mid-session (restart with the flag).
- **Prefer `auto` mode** for far fewer prompts *with* background safety checks.

Start it (in a sandbox only):

```
claude --permission-mode bypassPermissions   # or --dangerously-skip-permissions
```

1.10 Managing Sessions

A session is a saved conversation tied to a project directory, stored locally as you work.

Command	What it does
`claude --continue`	Resume the most recent session in this directory
`claude --resume`	Open the session picker to browse/search
`claude --resume <name>`	Resume a named session directly
`/rename <name>`	Name the current session so it's findable
`/branch <name>`	Fork the conversation to try a different approach
`/clear` · `/compact` · `/context`	Manage context within a session

Run parallel sessions on different branches with git worktrees (`claude --worktree <name>`).

1.11 Scheduled Tasks (Desktop Routines)

Scheduled tasks start a new Claude session automatically — great for daily code reviews, dependency audits, or morning briefings.

1. In the Desktop app, click **Routines** → **New routine** → **Local**.
2. Set a **Name**, **Instructions** (what Claude should do), working folder, and permission mode.
3. Pick a **Schedule**: Manual, Hourly, Daily, Weekdays, or Weekly — or just describe it ("every 6 hours").
4. **Run now** once and approve any permission prompts so future runs don't stall.

Local tasks run only while the app is open and your machine is awake. For runs that fire even when your computer is off, use cloud **Routines**.

> Tip: you can create one by asking in plain language — "set up a daily code review that runs every morning at 9am."

1.12 Keep Working Toward a Goal (`/goal`)

`/goal` sets a completion condition and Claude keeps working across turns until it's met. After each turn, a small fast model (Haiku) checks the condition; the goal clears automatically once satisfied.

Command	What it does
`/goal <condition>`	Set a verifiable end state and start working toward it
`/goal`	Check status: turns, tokens, and the latest reason
`/goal clear`	Remove an active goal before it's met

Write conditions Claude can prove in the transcript, e.g. `all tests in test/auth pass and the lint step is clean`. Bound long runs with a clause like `or stop after 20 turns`.

Activity 1 — Build & Deploy a Website

Goal: Build a single-page responsive website for a software development company, then push it to GitHub and deploy it on GitHub Pages.

This activity runs in two parts: **Activity 1a** covers Steps 1–3 (Goal & Features → Plan → Execute/build), then we cover context engineering in class, and **Activity 1b** covers Steps 4–7 (Context `/init` → Commit → Deploy → CI/CD) and presenting your live GitHub Pages link.

Step-by-step

1. **Create a project folder** and open a terminal there:

   ```
   mkdir software-dev-site && cd software-dev-site
   ```

   Launch Claude Code (`claude`) or open the Claude Code panel in VS Code.

Complete the **7-step build workflow**:

2. **Step 1 — Goal & Features.** Draft a detailed build prompt (use ChatGPT to expand your idea), then paste the website prompt below into Claude Code.

3. **Step 2 — Plan Mode.** Press **Shift+Tab** to enter Plan mode and let Claude propose a plan; review it before approving.

4. **Step 3 — Execute (Bypass Permissions).** Approve the plan and let Claude build the site hands-free. To enable bypass-permissions mode in VS Code: Settings → Extensions → Claude Code → enable **"Allow dangerously skip permissions"**, then pick **Bypass permissions** in the prompt-box mode selector (CLI: `claude --dangerously-skip-permissions`). Use this only in a trusted/sandboxed project.

5. **Step 4 — Context Engineering (`/init`).** Run `/init` to generate a `CLAUDE.md` so Claude remembers the project's structure and conventions. (See the example CLAUDE.md below.)

6. **Verify the build** — open `index.html` in a browser; submit the enquiry form empty (errors show) and valid (confirmation + data logged to console).

7. **Step 5 — Commit to GitHub:**

   ```
   git init && git add . && git commit -m "Initial website"
   gh repo create software-dev-site --public --source=. --push
   ```

8. **Step 6 — Deployment (GitHub Pages via GitHub Actions)** — ask Claude:

   Create a GitHub Actions workflow that deploys this site to GitHub Pages on every push to main, then enable Pages for the repo.

   Add a README, then visit `https://<your-username>.github.io/software-dev-site/`.

9. **Step 7 — CI/CD.** Push a small change and confirm the GitHub Action automatically rebuilds and redeploys the site.

Example `CLAUDE.md`

```
# CLAUDE.md

## What this is
```

```
Single-page marketing site. HTML + CSS + vanilla JS — no framework, no
build step.

## Running
Open index.html directly in a browser. No dev server.

## Architecture
- index.html  — sections: #home, #services, #testimonials, #contact
- styles.css  — design tokens in :root variables; mobile-first
- script.js   — enquiry-form validation (one DOMContentLoaded handler)
```

Reference repo: https://github.com/alfredang/softwaredevleopment

The Website Prompt

> **Build a single-page, responsive website for a software development company using HTML, CSS, and vanilla JavaScript only (no frameworks).** Output everything in one `index.html` file with inline `<style>` and `<script>`, or split into `index.html`, `styles.css`, and `script.js` — your choice.
>
> **Goal:** A modern, professional landing page that markets custom software development services and captures leads through an enquiry form.
>
> **Sections (in order):**
>
> **1. Hero Section** — Full-width banner with a strong headline (e.g. "We Build Software That Scales") and a one-line subheadline. Short supporting paragraph. Primary CTA button ("Get a Quote") that smooth-scrolls to the enquiry form. Clean gradient or solid background, large readable typography, centered content. Optional navbar with logo and anchor links (Home, Services, Testimonials, Contact).
>
> **2. Testimonial Section** — Section heading (e.g. "What Our Clients Say"). 3 testimonial cards, each with: client quote, client name, role/company, and avatar placeholder (initials circle or image). Responsive grid: 3 columns on desktop, 1 column on mobile. Subtle card shadows and hover effect.
>
> **3. Enquiry Form Section** — Section heading (e.g. "Get in Touch"). Fields: Name (required), Email (required), Phone (optional), Service interested in (dropdown), Message (textarea, required). Submit button. Submission handled in JavaScript: intercept submit with `event.preventDefault()`; validate fields (required checks + email regex) and show inline error messages; on success show a confirmation message ("Thanks! We'll be in touch shortly.") and reset the form; log the collected form data as an object to the console. Include a commented-out `fetch()` example showing how to POST to an API endpoint.
>
> **Styling Requirements:** Mobile-first and fully responsive (flexbox/grid + media queries). Consistent palette (CSS variables for primary, accent, background, text). Modern font stack or a Google Font. Smooth scrolling. Accessible: semantic HTML5, a `label` for every input, sufficient contrast.
>
> **Deliverable:** Working, copy-paste-ready code that runs by opening the file in a browser — no build step.

Topic 2 — Tools and Commands

2.0 The `.claude` Directory

Claude Code keeps its configuration in two places — your **user** folder (applies to every project) and the **project** folder (shared with your team via version control).

User scope — `~/.claude/`: `settings.json` (settings, hooks), `CLAUDE.md` (personal memory), `agents/`, `skills/`, `commands/`, `projects/<project>/` (session transcripts + `memory/`), and `plugins/`. MCP servers and per-project local config live in `~/.claude.json`.

Project scope — `<project>/.claude/`: `settings.json` (shared) and `settings.local.json` (personal, gitignored), `CLAUDE.md` and `rules/`, `commands/` (slash commands), `skills/<name>/SKILL.md`, `agents/<name>.md` (sub agents). Project MCP servers live in `.mcp.json` at the repo root.

User scope applies everywhere; project scope is committed so your whole team shares the same commands, skills, agents and settings.

User level vs project level — where each customisation lives. Every customisation can be stored at two levels. User level (~/.claude/) applies to ALL your projects; project level (<project>/.claude/) applies only to that ONE project and travels with it in git (slide 42):

Commands — user: ~/.claude/commands/<name>.md; project: <project>/.claude/commands/<name>.md.

MCP tools — user: ~/.claude.json (claude mcp add -s user ...); project: <project>/.mcp.json (claude mcp add -s project ...).

Skills — user: ~/.claude/skills/<name>/SKILL.md; project: <project>/.claude/skills/<name>/SKILL.md.

CLAUDE.md — user: ~/.claude/CLAUDE.md (your personal memory); project: <project>/CLAUDE.md (shared with the team).

Hooks — user: the "hooks" block in ~/.claude/settings.json; project: the "hooks" block in <project>/.claude/settings.json.

Keep personal preferences at user level; commit team-shared rules, commands, skills and hooks at project level. Both levels load together — where they conflict, project settings take precedence.

2.1 Custom Commands

Custom commands are reusable prompts saved as Markdown files. Typing a slash command runs a multi-step workflow the same way every time.

- **Location:** `.claude/commands/` in your project (or `~/.claude/commands/` for personal commands).
- **Create one:** make `name.md` — the filename becomes `/name`.
- **Reload:** restart Claude Code to pick up new commands.
- **Arguments:** use `$ARGUMENTS` to pass input into the command.

2.2 MCP Tools

The **Model Context Protocol (MCP)** is an open standard that lets Claude Code connect to external tools and data sources through a common interface.

- **MCP Server** — exposes tools/data (e.g. Playwright, GitHub, databases).
- **MCP Client** — Claude Code connects to servers and calls their tools.
- **Transport** — stdio or HTTP/SSE; transport-agnostic.
- **Tools** — each server publishes callable actions Claude can use.

Playwright MCP lets Claude drive a real browser — open pages, click, fill forms and capture screenshots.

Connecting a server. Register a server with `claude mcp add`, then verify with `claude mcp list` and manage in-session with `/mcp`. Servers are saved at one of three scopes:

Scope	Stored in	Available to
`local` (default)	`~/.claude.json` (this project)	Only you, this project
`project`	`.mcp.json` in the repo root	Everyone who clones the repo
`user`	`~/.claude.json` (top level)	Only you, all projects

```
# hosted (HTTP) server
claude mcp add --transport http docs https://code.claude.com/docs/mcp

# local (stdio) server, shared with the team via .mcp.json
claude mcp add playwright -s project -- npx -y @playwright/mcp@latest

claude mcp list
```

Activity 2 — Create a `/gitpush` Custom Command

Goal: Create a custom command that automates pushing, documenting, deploying and securing your project.

Step-by-step

1. **Create the commands folder** in your project:

   ```
   mkdir -p .claude/commands
   ```

2. **Create the file** `.claude/commands/gitpush.md` with the following content:

   ```
   ---
   description: Push code to GitHub, create README, deploy Pages,
   secure the repo
   ---

   Perform the following steps for this project:

   1. Push/update all the code to GitHub (commit any changes first).
   2. Create or update a README that documents the project.
   ```

```
3. Create a GitHub Page via GitHub Actions.
4. Update the repository "About" section and add the GitHub Pages
link.
5. Run a security scan to make sure no sensitive data (API keys,
secrets,
   passwords, personal data) is published to the internet.

Target repository / notes: $ARGUMENTS
```

3. **Restart Claude Code** so the new command is detected.
4. **Run the command:**

   ```
   /gitpush software-dev-site
   ```

5. **Verify** the repo has a README, GitHub Pages is live, the About section shows the Pages link, and the security scan reported no leaked secrets.

Activity 3 — Install & Use Playwright MCP

Goal: Install the Playwright MCP tool at the project level, screenshot your website and push it to GitHub.

Step-by-step

1. **Install Playwright MCP at the project level:**

   ```
   claude mcp add playwright -s project npx @playwright/mcp@latest
   ```

2. **Restart Claude Code** and approve the browser permissions when prompted.
3. **Confirm the server is connected:**

   ```
   /mcp
   ```

4. **Ask Claude to screenshot your site:**

   Use the Playwright MCP tool to open my deployed website (https://.github.io/software-dev-site/) and capture a full-page screenshot. Save it as screenshot.png.

5. **Add the screenshot to the README:**

   Add screenshot.png to the README under a “Preview” heading, then commit and push to GitHub.

6. **Verify** the screenshot renders correctly in your GitHub README.

Activity 3B — Test the Enquiry Form with Playwright

Goal: Use the Playwright MCP tool to test the enquiry form end-to-end and confirm submissions reach your email.

Step-by-step

1. **Wire the form to email.** Ask Claude to send form submissions to **angch@tertiaryinfotech.com** — for example using a form service such as Formspree, or a `fetch()` POST to your endpoint:

   Connect the enquiry form so submissions are emailed to angch@tertiaryinfotech.com (use Formspree or a fetch to an endpoint).

2. **Open the site with Playwright:**

   Use the Playwright MCP tool to open my deployed site (https://.github.io/software-dev-site/).

3. **Fill and submit the form** with test data:

   Fill the enquiry form with name “Test User”, email “test@example.com”, a sample message, then submit it.

4. **Verify the on-page result** — confirm the success message (“Thanks! We’ll be in touch shortly.”) appears and the form data is logged/captured.

5. **Check your inbox** — confirm the enquiry email actually arrives at angch@tertiaryinfotech.com.

6. **Re-run** the test after any change so every enquiry reliably reaches your email, then commit and push.

Topic 3 — Skills, Agents & Hooks

3.1 Agent Skills

Skills are reusable folders of instructions (`SKILL.md`) plus optional scripts and resources that teach Claude a specialised task. Claude loads a skill only when it is relevant.

- **`SKILL.md`** — name, description and instructions Claude follows on demand.
- **Project skills** — live in `.claude/skills/`, shared with your repo and team.
- **Install** — `npx skills add <repo> --skill <name>`.
- **Customise** — edit the skill to fit your project.

SKILL.md vs CLAUDE.md. Both are Markdown instruction files, and both tell Claude how you want work done. The difference is when Claude loads them — and that difference decides which one you should reach for.

SKILL.md — loaded on demand	CLAUDE.md — loaded every session
Trigger: Claude reads the skill's name + description and loads the body only when the task matches.	Trigger: read automatically at the start of every session — always in context.
Scope: one specialised capability — a workflow, an output format, a house standard.	Scope: project-wide facts — stack, conventions, commands to run, things not to do.

Cost: costs no context until it fires, so you can keep dozens installed.

Lives in: .claude/skills/<name>/SKILL.md (or ~/.claude/skills/ for all projects).

Cost: spends context on every single turn, so keep it tight and high-value.

Lives in: CLAUDE.md at the project root (or ~/.claude/CLAUDE.md for all projects).

Rule of thumb: put stable project facts in CLAUDE.md; put specialised, occasional procedures in a skill. If you find yourself writing "when I ask you to do X, follow these ten steps" in CLAUDE.md, that is a skill trying to escape.

3.2 Sub Agents

Sub agents are specialised assistants with their own prompt, tools and context window. Delegate focused jobs — like security review or UX critique — to keep the main thread clean.

- **Create** with `/agents`, then choose **Project** to save under `.claude/agents/`.
- **Give a clear role** — describe purpose, tools and when to invoke it.
- **Isolated context** — each sub agent runs in its own context window.

Why use sub agents? Reach for one when a side task would flood your main conversation with search results, logs or file contents you won't reference again. Sub agents help you:

- **Preserve context** — exploration and logs stay in the sub agent's own window; only the summary returns to your main conversation.
- **Enforce constraints** — limit exactly which tools the sub agent may use.
- **Reuse & specialise** — reuse configurations across projects (user-level agents) and give each a focused system prompt for its domain.
- **Control costs** — route narrow tasks to a faster, cheaper model.

Define a *custom* sub agent when you keep spawning the same kind of worker with the same instructions.

3.3 Hooks

Hooks are shell commands that run automatically at defined points in Claude Code's lifecycle.

Event	When it runs
`PreToolUse`	Before a tool — validate or block an action.
`PostToolUse`	After a tool — auto-format or test.
`Notification`	When Claude needs input or finishes.
`Stop`	When a task ends — cleanup or checks.

Hooks are configured in `.claude/settings.json` under `"hooks"`.

Automating with hooks. Hooks are *deterministic* — they always run at their event rather than relying on the model to choose to. Common uses: auto-format after edits, run tests, block risky commands, or send notifications. Add a `"hooks"` block with a matcher and a command; hooks receive event context on stdin as JSON. Example — format files after every edit:

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
```

```
      "hooks": [{ "type": "command", "command": "npx prettier --
write ." }]
    }]
  }
}
```

Activity 4 — Install & Run Skills

Goal: Install and run two skills to improve your website's design and SEO.

Step-by-step

1. **Install the frontend-design skill** at the project level:

   ```
   npx skills add https://github.com/anthropics/skills --skill
   frontend-design
   ```

   (This installs into `.claude/skills/`.)

2. **Customise the frontend design** — ask Claude:

   ```
   Using the frontend-design skill, redesign my website to look modern
   and
   professional and appeal to software-development clients. Improve
   the hero,
   testimonials and form styling while keeping it responsive.
   ```

3. **Install the SEO audit skill:**

   ```
   npx skills add https://github.com/coreyhaines31/marketingskills --
   skill seo-audit
   ```

4. **Run the SEO audit:**

   ```
   Run the seo-audit skill on my website and apply the recommended
   changes
   (title tags, meta description, headings, alt text, Open Graph,
   etc.).
   ```

5. **Commit and push** the improvements to GitHub.

Activity 5 — Create Custom Sub Agents

Goal: Create two project sub agents — one for security, one for UI/UX & lead optimisation.

Step-by-step

1. **Create the security sub agent.** Run:

   ```
   /agents
   ```

Choose **Create new agent** → **Project**, then describe it:

A security review agent that scans this website for vulnerabilities (XSS, exposed secrets, insecure form handling, missing headers) and recommends hardening measures.

This saves to `.claude/agents/`.

2. **Run the security agent** and apply its recommended fixes.
3. **Install the lead-magnets skill:**

```
npx skills add https://github.com/coreyhaines31/marketingskills --skill lead-magnets
```

4. **Create a UI/UX sub agent** via `/agents` (Project) with this role:

An expert UI/UX engineer that reviews the website and, using the lead-magnets skill, optimises it for lead capture and conversion.

5. **Run the UI/UX agent**, apply improvements, then commit and push.

Activity 6 — Hooks (Floating WhatsApp Widget)

Goal: Add a floating WhatsApp widget to the website from Activity 1, and use a Claude Code hook so it auto-opens after 10 seconds with suggested queries and voice activation.

Claude Code hooks are shell commands that fire on Claude's tool lifecycle (configured in .claude/settings.json, with scripts kept under .claude/hooks/). You don't write them by hand — just ask Claude Code, in plain language, to create them. The widget behaviour itself lives in script.js on the site.

Step-by-step

Just ask Claude Code, in plain text, for each of these:

1. **Add the widget —** ask Claude Code to add a floating WhatsApp widget fixed to the bottom-right of every page: a round WhatsApp button that expands into a small chat panel when clicked.
2. **Auto-open after 10 seconds —** ask Claude to add a Claude Code hook (under .claude/hooks/, wired in .claude/settings.json) plus the page logic so the widget opens by itself 10 seconds after the page loads.
3. **Prompt and suggested queries —** on auto-open, greet the visitor with "Hi! How can I help you?" and show 3–4 suggested queries as quick-reply chips (for example: course fees, schedule, funding). Add voice activation with the Web Speech API so the visitor can speak their question instead of typing.
4. **Test with Playwright —** ask Claude to use the Playwright MCP tool to load the page, wait 10 seconds, and confirm the widget auto-opens with the greeting, the suggested queries and the voice-input control.

Activity 7 — Test, Convert & Present

Goal: Test the enquiry form, add a WhatsApp widget, run final checks and present your project.

Step-by-step

1. **Test the enquiry form with Playwright MCP:**

   Use the Playwright MCP tool to fill in and submit the enquiry form with test data, and confirm the success message appears and the data is captured.

2. **Route enquiries to your email.** Ask Claude to wire the form so submissions are sent to **angch@tertiaryinfotech.com** (e.g. via a form service such as Formspree, or a commented `fetch()` to your endpoint). Test that an enquiry arrives in the inbox.

3. **Add a WhatsApp floating widget** — ask Claude:

   Add a WhatsApp floating widget fixed to the bottom-right of every page. On click it opens a chat to +65 9698 3731 with suggested starter queries (e.g. "I'd like a quote", "Tell me about your services").

   The link format is: `https://wa.me/6596983731?text=<suggested message>`.

4. **Run a final pass** — re-run the security sub agent and the lead-magnet UI/UX sub agent, and apply any final improvements.

5. **Deploy the final version** — run `/gitpush` to push, update the README and redeploy GitHub Pages.

6. **Prepare your capstone presentation.** Cover:

   - What you built and the live URL
   - The agentic loop and context techniques you used
   - Your `/gitpush` custom command
   - MCP tools (Playwright) and what you tested
   - Skills, sub agents and hooks you added
   - Security and lead-conversion improvements

7. **Present to the class.**

Capstone Project

Goal: Create your **own new project** with Claude Code using the **7-step workflow**, then present it. Your project can be a **website, a web tool, a full-stack app, or a mobile app**.

Draft your build prompt

Not sure how to phrase it? Ask a chatbot (ChatGPT or Claude) to write your build prompt first, then paste the result into Claude Code:

```
Create a prompt to build a web app for [your idea]
with the following features:
  - [feature 1]
  - [feature 2]
  - [feature 3]
It should be built in HTML / CSS / JavaScript.
Output the prompt in markdown format.
```

Step-by-step

1. **Pick an idea** — decide what to build (website, web tool, full-stack or mobile app).
2. **Step 1 — Goal & Features.** Draft a clear, detailed build prompt (use ChatGPT to help expand the idea).
3. **Step 2 — Plan Mode.** Let Claude plan the build (Shift+Tab) and review the plan.
4. **Step 3 — Execute (Bypass Permissions).** Approve the plan and let Claude build it.
5. **Step 4 — Context (`/init`).** Generate a `CLAUDE.md` for your project.
6. **Step 5 — Commit to GitHub.** Initialise git and push your code.
7. **Step 6 — Deployment.** Deploy your app (e.g. GitHub Pages for web projects).
8. **Step 7 — CI/CD.** Automate redeploys with GitHub Actions.
9. **Present** — demo your project and walk through how you built it with Claude Code.

Key Concepts Summary — What Triggers What

Claude Code can be extended in six ways. They differ mainly in what makes Claude load them: some are always present, some are pulled in only when relevant, some you fire yourself, and some are fired by an event or by another agent. Choosing the right mechanism is mostly a question of matching the trigger to the job.

Mechanism	Trigger — when it loads / runs
CLAUDE.md	Every session. Project memory, read automatically at the start of each session, so it is always in context. Generate with /init and keep it short and high-signal.
Skills (SKILL.md)	On demand. Claude matches the skill's name and description against the task and loads it only when relevant — so it costs no context until it fires.
Tools (built-in & MCP)	On demand. Claude invokes tools itself during the agentic loop whenever the job needs them — reading files, searching, running commands, driving a browser via MCP.
Slash commands	Manually. Reusable prompts saved in .claude/commands/; they run only when you type /<name> yourself. Use them for multi-step workflows you repeat.
Hooks	By event. Shell commands configured in .claude/settings.json that fire deterministically at lifecycle events (PreToolUse, PostToolUse, Notification, Stop) — they always run, rather than relying on the model to choose to.

Sub agents	By delegation. Specialised assistants with their own prompt, tools and context window, spawned when the main agent delegates a focused job and reports only the summary back.

Why it matters: anything loaded every session competes for the context window. Keep CLAUDE.md tight, and push specialised or occasional procedures out into skills, commands, hooks and sub agents — so Claude carries only what the current task actually needs.

Lab Materials

The hands-on labs are maintained in their own repository so you can clone them and keep working after class. Each lab folder contains a README that indexes its parts, and every step gives you the exact prompt to paste, what you should see, and a checkpoint to confirm before moving on.

Lab	What you build
Lab 1 — labs/lab1-seven-steps/	Topic 1. The 7-step build workflow end to end: goal & features, plan mode, execute, /init context, commit to GitHub, GitHub Pages deployment and CI/CD. Includes a finished reference site and three sample sites (bridal booking, interior design, lead generator).
Lab 1b — labs/lab1b-commands-mcp/	Topic 2. Build a /publish custom slash command that scans for secrets, pushes, deploys and updates the README; install Playwright MCP and use it to screenshot the live site into the README, then fold that step back into the command.
Lab 2 — labs/lab2-skills/	Topic 3. Install the cybersecurity-analyst, agent-kanban and ui-ux-pro-max skills at project level, then use them to harden, plan and redesign the Lab 1 website in a green visual identity.
Lab 3 — labs/lab3-hooks/	Topic 3. A Stop hook that pops a congratulations dialog, the floating WhatsApp widget with a guard hook, an add-task sub agent, a security-scan sub agent that writes security-scan.json, and /loop to re-scan on a schedule.

Labs repository: https://github.com/tertiarycourses/TGS-2025052468-Claude-Code

Quick Command Reference

Command	What it does
`claude`	Launch Claude Code
`claude --continue` / `--resume`	Resume the last / a chosen session
`/init`	Generate `CLAUDE.md` project memory
`/memory`	Browse & edit CLAUDE.md and auto memory
`/context`	See what's using the context window

`/compact`	Summarise conversation to save context
`/clear`	Clear context for a new task
`@file`	Mention a file/folder to add to context
`Shift+Tab`	Cycle permission modes (default → acceptEdits → plan)
`/rename` · `/branch`	Name / fork the current session
`/goal <condition>`	Keep working until a condition is met
`/mcp`	View & manage connected MCP servers
`/agents`	Create / manage sub agents
`/gitpush`	Your custom push-deploy-secure command
`claude mcp add <name> -s project <cmd>`	Add an MCP server at project level
`npx skills add <repo> --skill <name>`	Install a skill

Support

If you have any enquiries during or after the class, contact us:

- **Email:** enquiry@tertiaryinfotech.com
- **Tel:** +65 6100 0613

Courseware & assessment: Download the courseware and complete the Written Assessment (WA) and Practical Performance (PP) on the LMS at https://lms-tms.tertiaryinfotech.com/.

Assessment flow (follow in order at the assessment step):

1. Complete the **TRAQOM** survey — scan the TRAQOM QR code on the LMS.
2. Take the **Assessment Digital Attendance**.
3. Complete the **Assessment** (WA + PP).
4. **Submit your assessment answers** on the LMS (https://lms-tms.tertiaryinfotech.com/).
5. **Sign the Assessment Summary Record**.

Remember to complete your AM, PM and Assessment digital attendance, and the certification & TRAQOM survey at ai-lms-tms.tertiaryinfo.tech.